UNIVERSITÀ degli STUDI di CATANIA
DIPARTIMENTO DI MATEMATICA E INFORMATICA

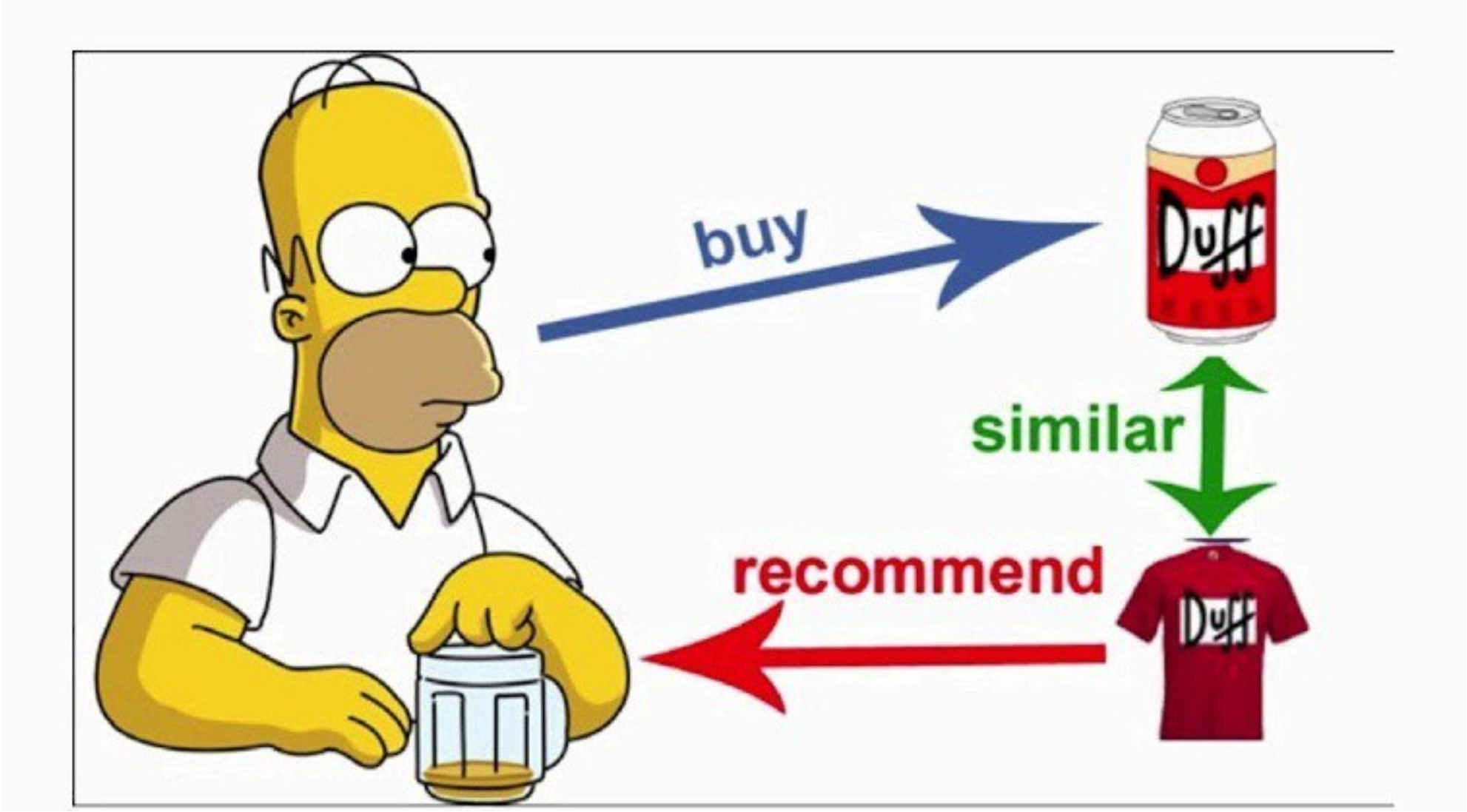
# Social Media Data Analysis 2023/2024

## *Recommendation Systems*

Francesco Ragusa
francesco.ragusa@unict.it
https://iplab.dmi.unict.it/ragusa/

IMAGE PROCESSING LABORATORY
https://iplab.dmi.unict.it/fpv/

# Recommendation Systems



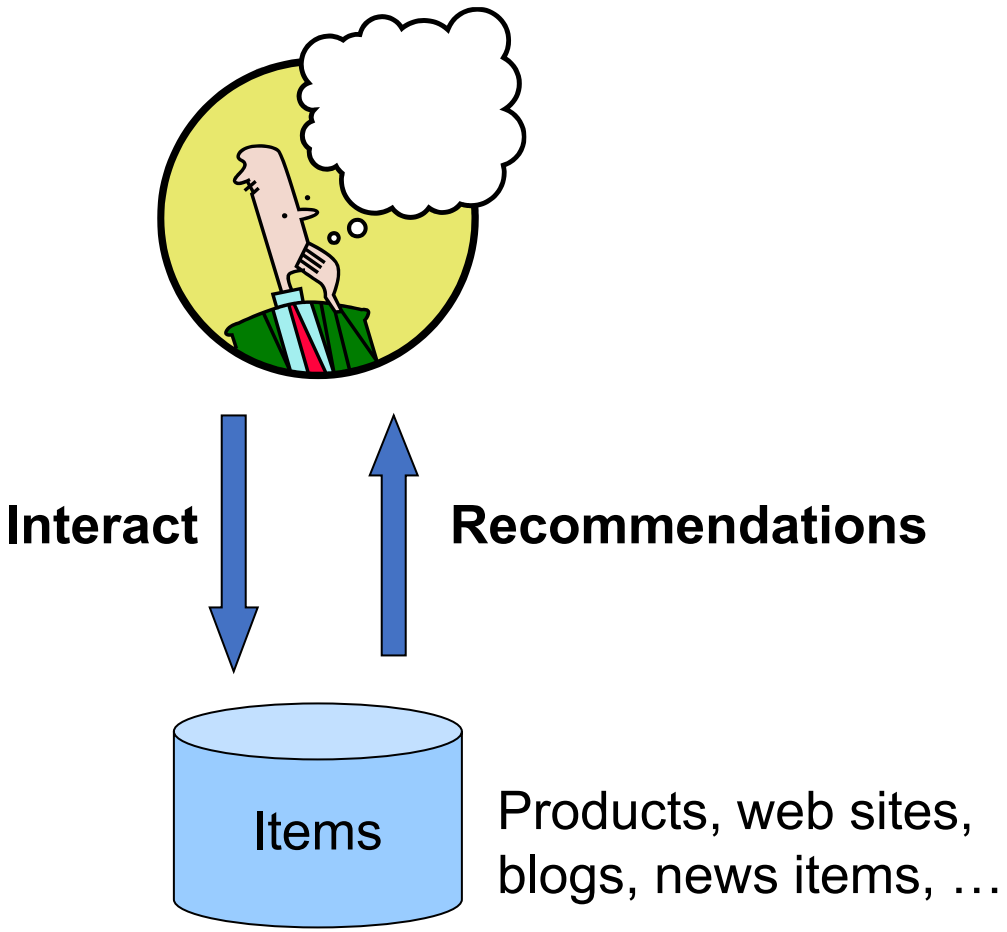
# Recommendation Systems

Every minute people upload 500 hours of videos

Users can listen to more than 80 million song tracks and podcasts

Users can buy more than 350 million different products

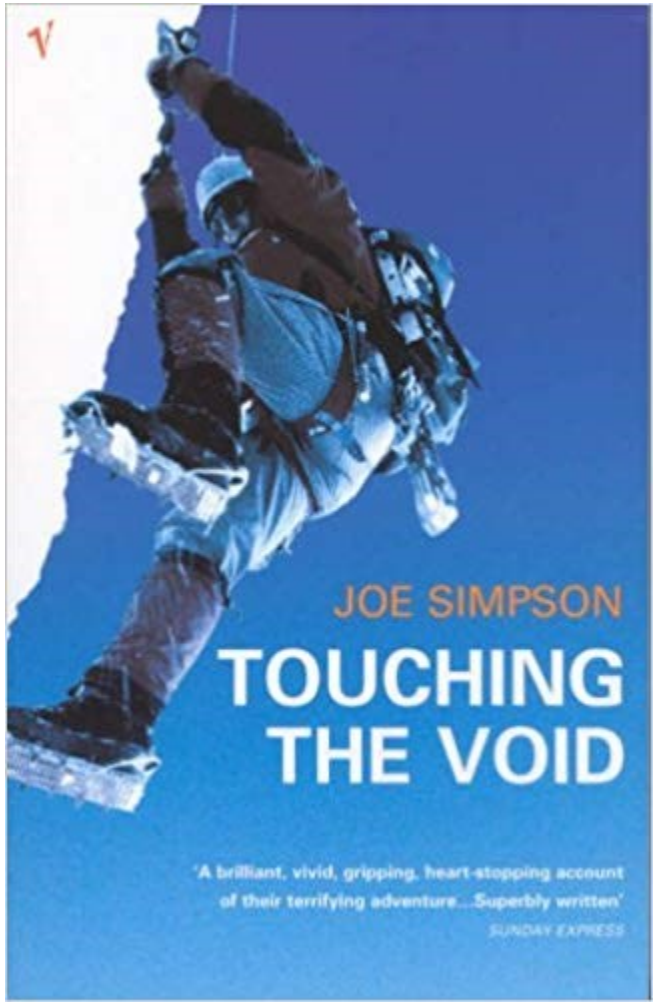
# Recommendation Systems




**Examples**

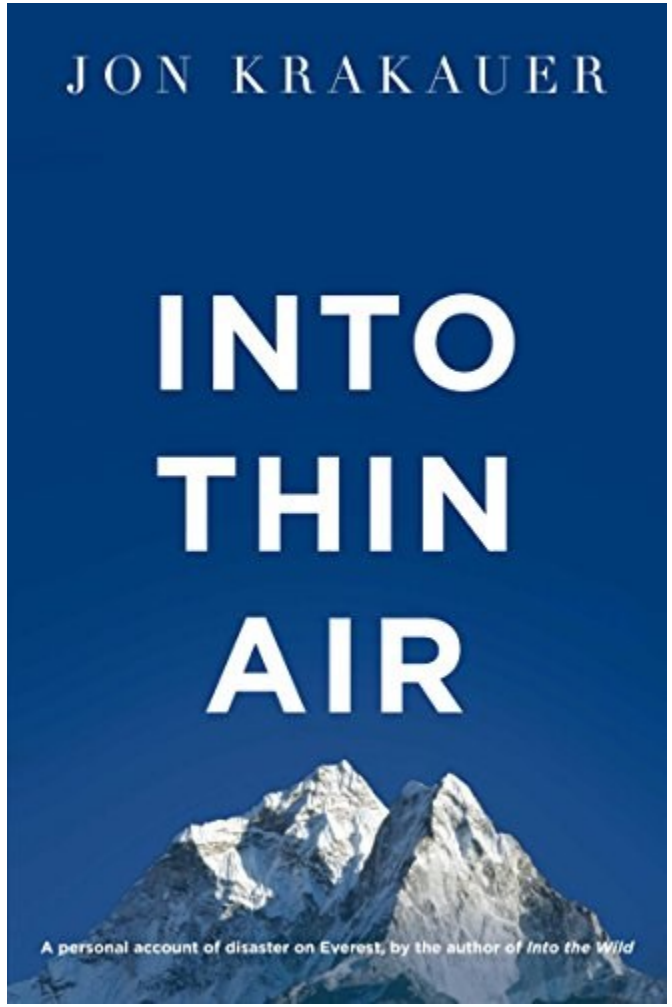
amazon.com
PANDORA
StumbleUpon
NETFLIX
del.icio.us
movielens
helping you find the *right* movies
last·fm
the social music revolution
Google News
YouTube
XBOX LIVE

# The power of recommendation




**1988**

**«You may also like»**




**1997**

# Types of Recommendations

## Editorial and hand curated:

- List of favorites
- Lists of “essential” items

## Simple aggregates:

- Top 10, Most Popular, Recent Uploads

## Tailored to individual users:

- Amazon, Netflix, Apple Music..

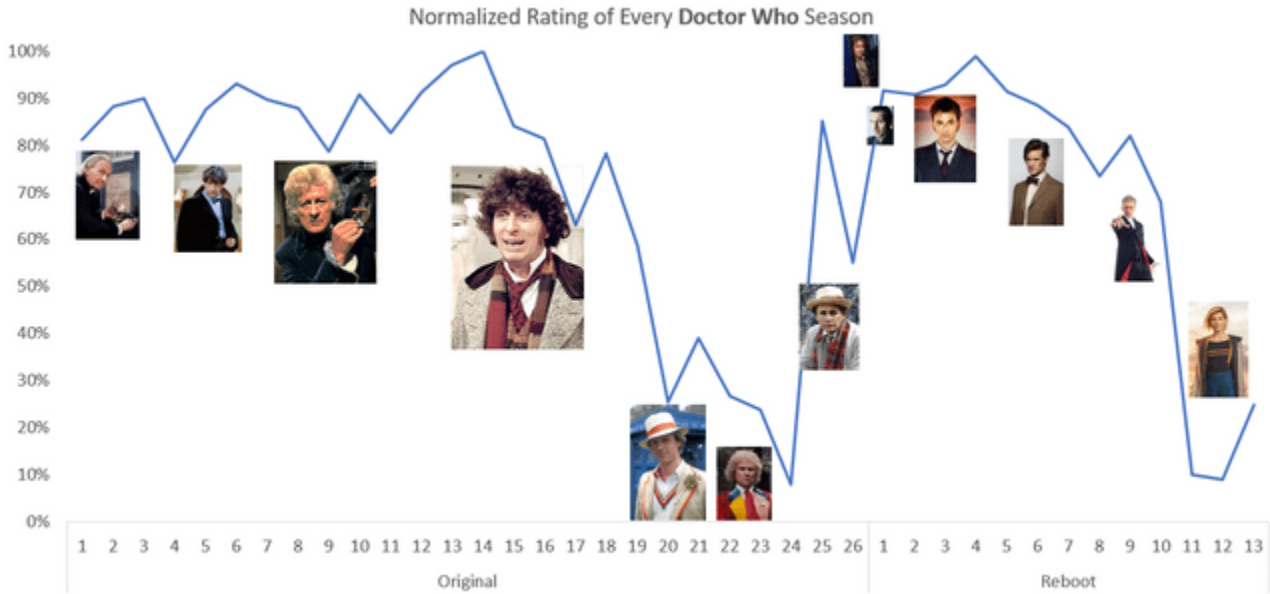
# Problem Definition

$X$: set of **users**

$S$: set of **items**

Ideally, we want to find: $u: X \times S \rightarrow R$ (**utility function**)

$R$: set of ratings ($\{1, \ldots, 5\}$, $[0,1]$, $\{0,1\}$)



# Problem Definition

Utility Matrix: $U = \left[r\left(x_i, s_j\right)\right]_{i,j}$ (sparse), $r\left(x_i, s_j\right) \in R$

| | | Harry Potter | | | Twilight | Star Wars | | |
|---|---|---|---|---|---|---|---|---|
| | | HP1 | HP2 | HP3 | TW | SW1 | SW2 | SW3 |
| Anita | $A$ | 4 | | | 5 | 1 | | |
| Beyonce | $B$ | 5 | 5 | 4 | | | | |
| Calvin | $C$ | | | | 2 | 4 | 5 | |
| David | $D$ | | 3 | | | | | 3 |

Will user A like Star Wars 2?

# Key Problems

## 1. Populating the utility matrix;

| | HP1 | HP2 | HP3 | TW | SW1 | SW2 | SW3 |
|---|---|---|---|---|---|---|---|
| $A$ | 4 | | | 5 | 1 | | |
| $B$ | 5 | 5 | 4 | | | | |
| $C$ | | | | 2 | 4 | 5 | |
| $D$ | | 3 | | | | | 3 |

Need some initial values (**cold start problem**)

## 2. Extrapolating unknown ratings from known ones;

| | HP1 | HP2 | HP3 | TW | SW1 | SW2 | SW3 |
|---|---|---|---|---|---|---|---|
| $A$ | 4 | | | 5 | 1 | | |
| $B$ | 5 | 5 | 4 | | | | |
| $C$ | | | | 2 | 4 | 5 | |
| $D$ | | 3 | | | | | 3 |

**Content-Based Systems**

**Collaborative Filters**

How to predict if a user will like an item?

# Key Problems

3. Evaluating extrapolation methods

# Populating the Utility Matrix

| | HP1 | HP2 | HP3 | TW | SW1 | SW2 | SW3 |
|---|---|---|---|---|---|---|---|
| $A$ | 4 | | | 5 | 1 | | |
| $B$ | 5 | 5 | 4 | | | | |
| $C$ | | | | 2 | 4 | 5 | |
| $D$ | | 3 | | | | | 3 |

- **Explicit**
  - Ask people to rate items
  - Doesn't work well in practice – people can't be bothered too much
  - Crowdsourcing: Pay people to label items
- **Implicit**
  - Learn ratings from user actions
    - E.g., purchase implies high rating

# Extrapolating Utilities

- Key problem: Utility matrix U is sparse
  - Most people have not rated most items
- The cold start problem
  - New items have no ratings
  - New users have no history

**Content-based**
**Collaborative Filtering**

# Content-based vs. Collaborative Filtering

**Database**

- Ella Fitzgerald: Jazz, Mid-20th century, vocal legend, famous duets, …
- Louis Armstrong: Jazz, Mid-20th century, vocal legend, famous duets, …

**content-based**

**Suggest**

**L. Armstrong**

**Customer W**

- Plays Ella Fitzgerald
- What should we recommend next?

**Customer D**

- Plays Ella Fitzgerald
- Plays Louis Armstrong

**collaborative filtering**

# Content-Based Recommendations

**Main idea:**

Recommend items to customer *x* similar to previous items rated highly by *x*

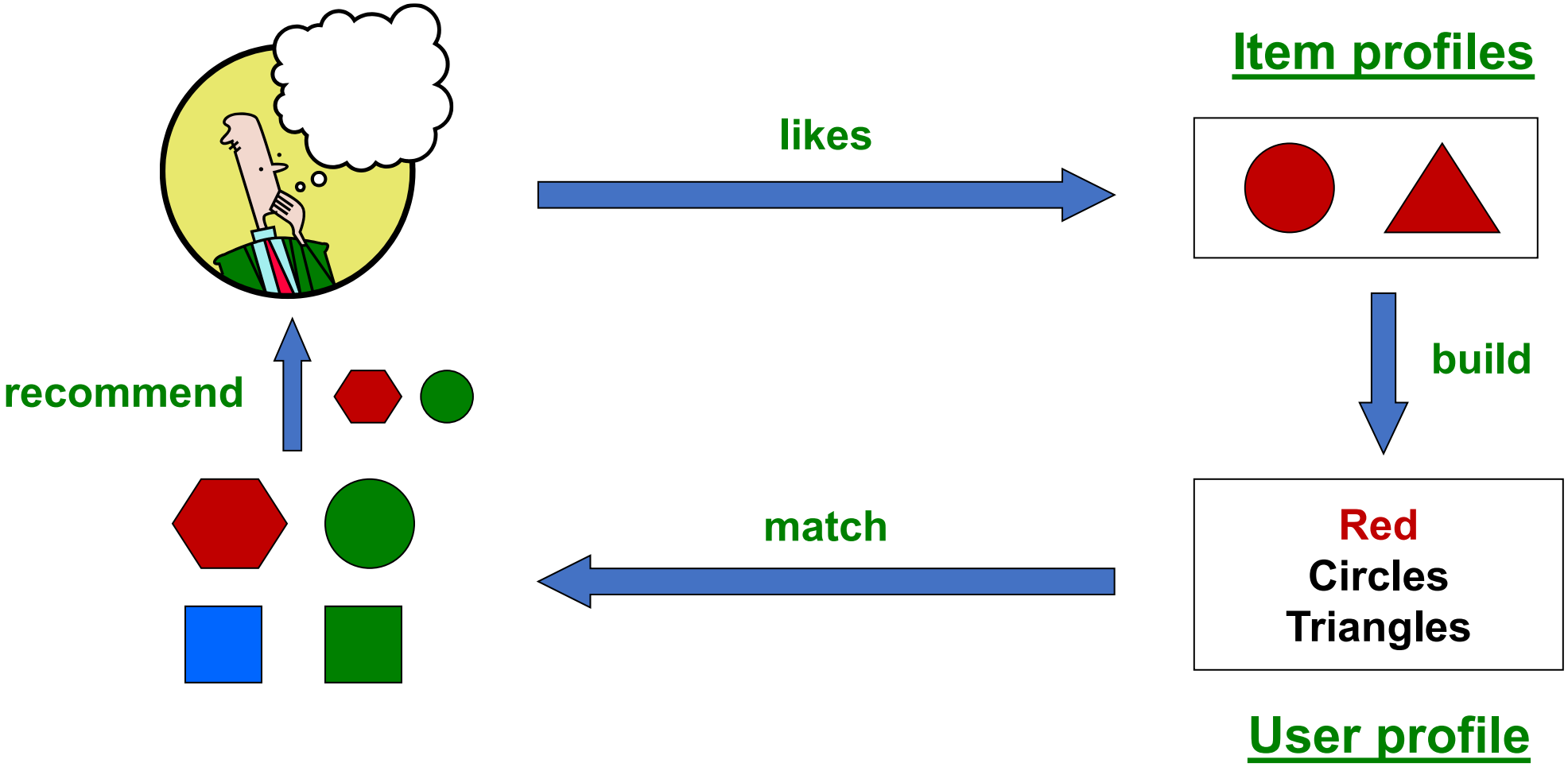
**Movie recommendations**

- Recommend movies with same actor(s), director, genre, …

**Websites, blogs, news**

- Recommend other sites with similar types or words

# Content-Based Recommendations



# Item Profiles – Item Characteristics

**identify the most important characteristics of a given item**

Movies (IMDB):
- Set of actors;
- Director;
- Year;
- Genre.

Books:
- Author;
- Genre;
- Year

CDs:
- Artist;
- Genre;
- Year

**can be entered by hand or crawled from some structured text (IMDB, reviews, textual descriptions)**

# Discovering Features

corpus

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun… It manages to be whimsical and romantic while laughing at the conventions of the fairy take genre.

TF-IDF over corpus
(assigns a weight to each word)

**Important Words**

Movie
Adventure
Satyrical
Romantic

[0 1 0 0 0 1 1 0 0 0 0]

feature vectors

[1 0 1 0 1 1 1 0 0 0 0 0]

Automatic Tagger

**Tags**

Sky, Grass, Solo, No humans

# Data Representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun… It manages to be whimsical and romantic while laughing at the conventions of the fairy take genre.

Bag of Words

**obtain a feature vector «directly»**

feature vectors

[0 1 0 0 0 … 1 1 0 0 0 0]

[1 0 1 0 1 1 … 1 0 0 0 0 0]

Bag of Visual Words

# Content-based Item Profiles

| | Melissa McCarthy | Actor A | Actor B | … | Johnny Depp | Comic Genre | Spy Genre | Pirate Genre |
|---|---|---|---|---|---|---|---|---|
| Movie X | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| Movie Y | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 |

**What if we want to have real or ordinal features too?**

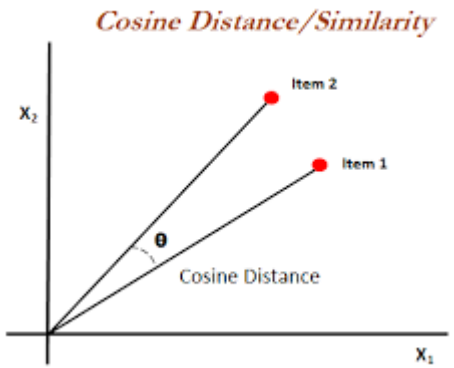
# Content-based Item Profiles

| | Melissa McCarthy | Actor A | Actor B | … | Johnny Depp | Comic Genre | Spy Genre | Pirate Genre | Avg Rating |
|---|---|---|---|---|---|---|---|---|---|
| Movie X | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 3 |
| Movie Y | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 4 |

# Content-based Item Profiles

| | Melissa McCarthy | Actor A | Actor B | … | Johnny Depp | Comic Genre | Spy Genre | Pirate Genre | Avg Rating |
|---|---|---|---|---|---|---|---|---|---|
| Movie X | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 3α |
| Movie Y | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 4α |

$$\text{cossim}(x, y) = \frac{x \cdot y}{||x|| \cdot ||y||} = \frac{\sum_i x_i y_i}{\sqrt{\sum_i x_i^2}\sqrt{\sum_i y_i^2}}$$



# Content-based Item Profiles

| | Melissa McCarthy | Actor A | Actor B | … | Johnny Depp | Comic Genre | Spy Genre | Pirate Genre | Avg Rating |
|---|---|---|---|---|---|---|---|---|---|
| Movie X | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | $3\alpha$ |
| Movie Y | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | $4\alpha$ |

$$\text{Cosine(Movie X, Movie Y)} = \frac{2+12\alpha^2}{\sqrt{5+9\alpha^2}\sqrt{5+16\alpha^2}}$$

$\alpha = 1$: 0.82 $\alpha = 2$: 0.94 $\alpha = 0.5$: 0.69

# Content-based User Profiles

**Want a vector with the same components/dimensions as items**

- Could be 1s representing user purchases
- Or arbitrary numbers from a rating

**User profile is aggregate of items:**

- Weighted average of rated item profiles

# Sample user profile

- Items are movies
- Utility matrix has 1 if user has seen movie
- 20% of the movies user U has seen have Melissa McCarthy
- U[“Melissa McCarthy”] = 0.2

| | Melissa McCarthy | Actor A | Actor B | … | |
|---|---|---|---|---|---|
| User U | 0.2 | .005 | 0 | 0 | … |

# Prediction

- Users and items have the same dimension

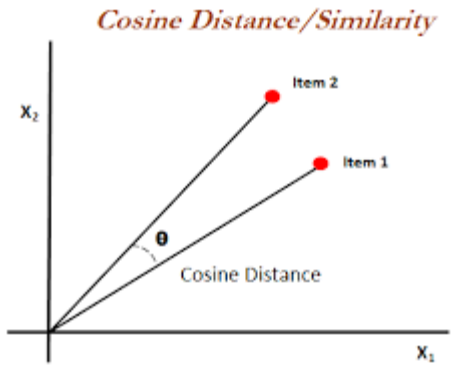
| | Melissa McCarthy | Actor A | Actor B | … | |
|---|---|---|---|---|---|
| Movie i | 0 | 1 | 1 | 0 | … |
| User x | 0.2 | .005 | 0 | 0 | 0 |

- Recommend the items whose vectors are most similar to the user vector

# HOW?

# Cosine Similarity

$$\text{cossim}(x, y) = \frac{x \cdot y}{||x|| \cdot ||y||} = \frac{\sum_i x_i y_i}{\sqrt{\sum_i x_i^2}\sqrt{\sum_i y_i^2}}$$




| | Melissa McCarthy | Actor A | Actor B | … | |
|---|---|---|---|---|---|
| **Movie i** | 0 | 1 | 1 | 0 | … |
| **User x** | 0.2 | .005 | 0 | 0 | 0 |

$$u(\boldsymbol{x}, \boldsymbol{i}) = \cos(\boldsymbol{x}, \boldsymbol{i}) = \frac{\boldsymbol{x} \cdot \boldsymbol{i}}{||\boldsymbol{x}|| \cdot ||\boldsymbol{i}||}$$

# Pros of Content-Based Recommendation Systems

- **No need for data on other users**
  - No user sparsity problems
- **Can recommend to users with unique tastes**
- **Can recommend new and unpopular items**
  - No first-rater problem
- **Can provide explanations**
  - «We are recommending this because you like action movies»

# Cons of Content-Based Recommendation Systems

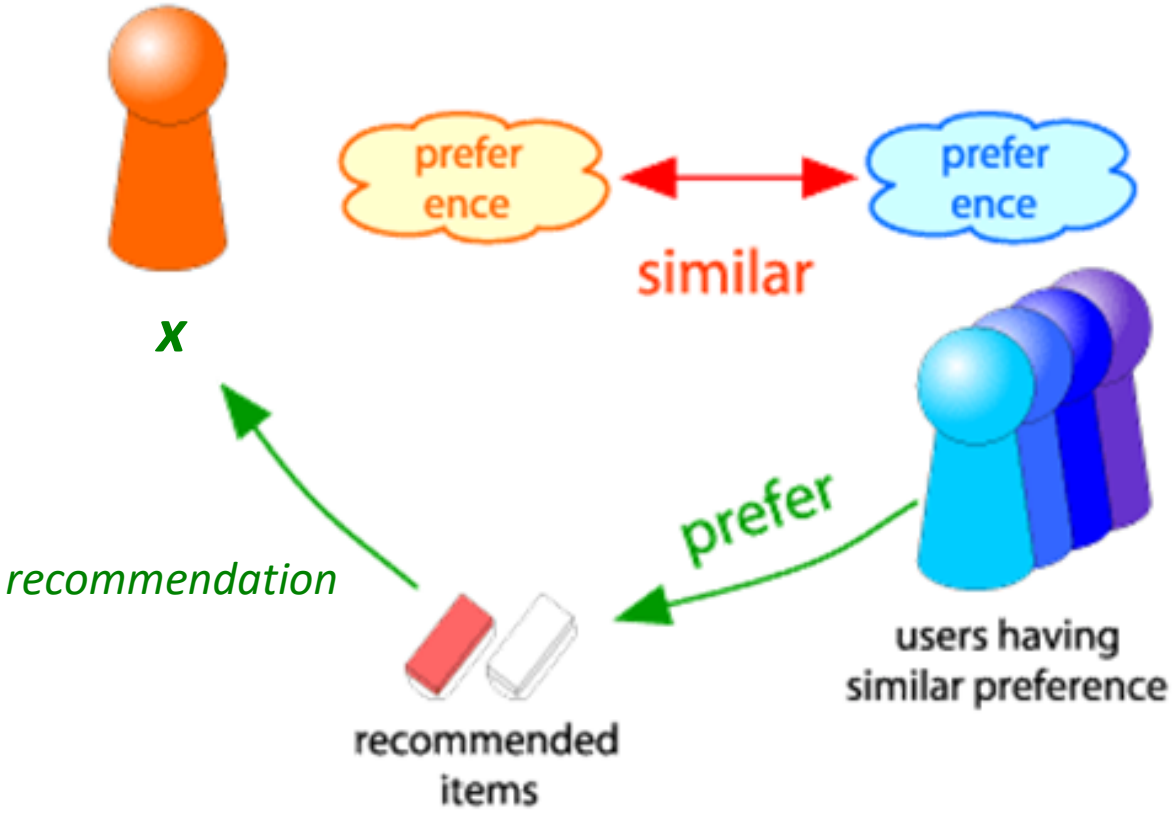
- **Finding the appropriate features is hard**
  - E.g., images, movies, music
- **Recommendations for new users**
  - **How to build a user profile?**
- **Overspecialization**
  - Tend to recommend items which are similar to already recommended ones;
  - People might have multiple interests;
  - Cannot exploit quality judgments of other users

# Collaborative Filtering

**Instead of using content features of items to determine what to recommend, find similar users and recommend items that they like!**

# Collaborative Filters: v1 User-User

- <u>Observation</u>: Similar users will like similar items;
- <u>Process</u>:
  - Consider user $x_i$;
  - Find a set $N$ of users whose ratings are "**similar**" to those of $x_i$ (they like similar items);
  - Estimate missing $x_i$'s ratings from the ratings of the users in $N$.

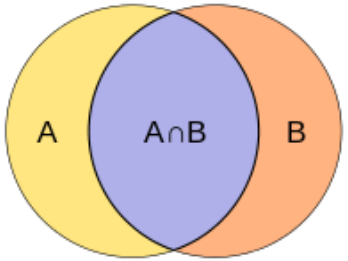


# Finding Similar Users – Binary Utilities

| | HP1 | HP2 | HP3 | TW | SW1 | SW2 | SW3 |
|---|---|---|---|---|---|---|---|
| $A$ | 1 | | | 1 | 1 | | |
| $B$ | 1 | 1 | 1 | 1 | 1 | | |
| $C$ | | | | 1 | 1 | 1 | |
| $D$ | | 1 | | | | | 1 |

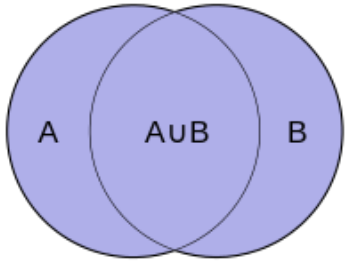
**Map each user to a set**

{HP1, TW, SW1}
{HP1, HP2, HP3, TW, SW1}
{TW, SW1,SW2}
{HP2, SW3}




Jaccard Similarity: $J(A,B) = \frac{|A \cap B|}{|A \cup B|}$

$\in [0,1]$




$$J(A,B) = \frac{3}{5};$$
$$J(A,C) = \frac{2}{4};$$
$$J(A,D) = 0;$$

# Finding Similar Users – Real Utilities

| | Harry Potter | | | Twilight | Star Wars | | |
|---|---|---|---|---|---|---|---|
| | HP1 | HP2 | HP3 | TW | SW1 | SW2 | SW3 |
| $A$ | 4 | | | 5 | 1 | | |
| $B$ | 5 | 5 | 4 | | | | |
| $C$ | | | | 2 | 4 | 5 | |
| $D$ | | 3 | | | | | 3 |

# Finding Similar Users – Real Utilities

$r_x$ = [*, _, _, *, ***]
$r_y$ = [*, _, **, **, _]

$r_x$ = {1, 0, 0, 1, 3}
$r_y$ = {1, 0, 2, 2, 0}

$$\text{sim}(\boldsymbol{x}, \boldsymbol{y}) = \cos(\boldsymbol{r}_x, \boldsymbol{r}_y) = \frac{r_x \cdot r_y}{||r_x||\ ||r_y||}$$

**Problem: This representation leads to unintuitive results**

# Problem with raw utility matrix cosine

| | Harry Potter | | | Twilight | Star Wars | | |
|---|---|---|---|---|---|---|---|
| | HP1 | HP2 | HP3 | TW | SW1 | SW2 | SW3 |
| $A$ | 4 | | | 5 | 1 | | |
| $B$ | 5 | 5 | 4 | | | | |
| $C$ | | | | 2 | 4 | 5 | |
| $D$ | | 3 | | | | | 3 |

Intuitively we want: sim(A, B) > sim(A, C)

$$\text{sim(A,B)} = \frac{4 \times 5}{\sqrt{4^2 + 5^2 + 1^2}\sqrt{5^2 + 5^2 + 4^2}} = 0.380$$

$$\text{sim(A,C)} = \frac{5 \times 2 + 1 \times 4}{\sqrt{4^2 + 5^2 + 1^2}\sqrt{2^2 + 4^2 + 5^2}} = 0.322$$

**0.380 > 0.322**

# Problem with raw utility matrix cosine

| | HP1 | HP2 | HP3 | TW | SW1 | SW2 | SW3 |
|---|---|---|---|---|---|---|---|
| $A$ | 4 | | | 5 | 1 | | |
| $B$ | 5 | 5 | 4 | | | | |
| $C$ | | | | 2 | 4 | 5 | |
| $D$ | | 3 | | | | | 3 |

$\mu_A = 10/3$

$\mu_B = 14/3$

$\mu_c = 11/3$

$\mu_D = 6/2$

| | HP1 | HP2 | HP3 | TW | SW1 | SW2 | SW3 |
|---|---|---|---|---|---|---|---|
| $A$ | $2/3$ | | | $5/3$ | $-7/3$ | | |
| $B$ | $1/3$ | $1/3$ | $-2/3$ | | | | |
| $C$ | | | | $-5/3$ | $1/3$ | $4/3$ | |
| $D$ | | 0 | | | | | 0 |

# Problem with raw utility matrix cosine

| | HP1 | HP2 | HP3 | TW | SW1 | SW2 | SW3 |
|---|---|---|---|---|---|---|---|
| $A$ | $2/3$ | | | $5/3$ | $-7/3$ | | |
| $B$ | $1/3$ | $1/3$ | $-2/3$ | | | | |
| $C$ | | | | $-5/3$ | $1/3$ | $4/3$ | |
| $D$ | | $0$ | | | | | $0$ |

$$\text{Cos(A,B)} = \frac{(2/3) \times (1/3)}{\sqrt{(2/3)^2 + (5/3)^2 + (-7/3)^2}\sqrt{(1/3)^2 + (1/3)^2 + (-2/3)^2}} = 0.092$$

$$\text{Cos(A,C)} = \frac{(5/3) \times (-5/3) + (-7/3) \times (1/3)}{\sqrt{(2/3)^2 + (5/3)^2 + (-7/3)^2}\sqrt{(-5/3)^2 + (1/3)^2 + (4/3)^2}} = -0.559$$

# Finding similar users with overlapping-item mean-centering

$r_x = \{1, 0, 0, 1, 3\}$
$r_y = \{1, 0, 2, 2, 0\}$

$r_x$ = [*, _, _, *, ***]
$r_y$ = [*, _, **, **, _]

## Mean-centering:

$\bar{r_x} = (1 + 1 + 3)/3 = 5/3$ $\bar{r_y} = (1 + 2 + 2)/3 = 5/3$

mean centered $r_x = \{-2/3, 0, 0, -2/3, 4/3\}$

## Keep only items they both rate:

$r_x = \{-2/3, \square, \square, -2/3, \square\}$
$r_x = \{-2/3, -2/3\}$

$r_y = \{-2/3, \square, \square, 1/3, \square\}$
$r_y = \{-2/3, 1/3\}$

# Mean-centered overlapping-item cosine similarity

**Instead of basic cosine similarity measure**

- $\text{sim}(\boldsymbol{x}, \boldsymbol{y}) = \cos(\boldsymbol{r}_x, \boldsymbol{r}_y) = \frac{r_x \cdot r_y}{||r_x||\ ||r_y||}$

**Mean-centered <u>overlapping-item</u> cosine similarity**

(Variant of Pearson correlation)

- $\boldsymbol{S_{xy}}$ = items rated by both users $\boldsymbol{x}$ and $\boldsymbol{y}$

$$\boldsymbol{sim}(\boldsymbol{x}, \boldsymbol{y}) = \frac{\sum_{s \in S_{xy}} (\boldsymbol{r}_{xs} - \overline{\boldsymbol{r}_x})(\boldsymbol{r}_{ys} - \overline{\boldsymbol{r}_y})}{\sqrt{\sum_{s \in S_{xy}} (\boldsymbol{r}_{xs} - \overline{\boldsymbol{r}_x})^2} \sqrt{\sum_{s \in S_{xy}} (\boldsymbol{r}_{ys} - \overline{\boldsymbol{r}_y})^2}}$$

# WHAT IS MISSING?

# Rating Predictions

**From similarity metric to recommendations for an unrated item *i*:**

Let $r_x$ be the vector of user *x*'s ratings
Let *N* be the set of *k* users most similar to *x* who have rated item *i*

## Prediction for item *I* of user *x*:

Rate *i* as the mean of what k-people-like-me rated *i*

$$r_{xi} = \frac{1}{k} \sum_{y \in N} r_{yi}$$

Even better: Rate *i* as the mean weighted by their similarity to me

$$r_{xi} = \frac{\sum_{y \in N} s_{xy}\, r_{yi}}{\sum_{y \in N} s_{xy}}$$

# Collaborative Filters: v2 Item-Item

For item *i*, find other similar items

Estimate rating for item *i* based on ratings for those similar items

Can use same similarity metrics and prediction functions as in user-user model

Rate *i* as the mean of my ratings for other items, weighted by their similarity to *i*

$$r_{xi} = \frac{\sum_{j \in N(i;x)} s_{ij} \; r_{xj}}{\sum_{j \in N(i;x)} s_{ij}}$$

***N(i;x)***…set of items rated by ***x*** *and* similar to ***i***

$s_{ij}$… similarity of items ***i*** and ***j***

$r_{xj}$…rating of user ***x*** on item ***j***

# Item-Item Collaborative Filtering - Example

users

movies

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | | 3 | | | 5 | | | 5 | | 4 | |
| 2 | | | 5 | 4 | | | 4 | | | 2 | 1 | 3 |
| 3 | 2 | 4 | | 1 | 2 | | 3 | | 4 | 3 | 5 | |
| 4 | | 2 | 4 | | 5 | | | 4 | | | 2 | |
| 5 | | | 4 | 3 | 4 | 2 | | | | | 2 | 5 |
| 6 | 1 | | 3 | | 3 | | | 2 | | | 4 | |

- unknown rating

- rating between 1 to 5

- We will see an example of item-item collaborative filter;
- We will consider a set of 2 neighbors $|N| = 2$ to predict ranking;

# Item-Item Collaborative Filtering - Example

users

movies

| | 1 | 2 | 3 | 4 | **5** | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | | 3 | | ? | 5 | | | 5 | | 4 | |
| 2 | | | 5 | 4 | | | 4 | | | 2 | 1 | 3 |
| 3 | 2 | 4 | | 1 | 2 | | 3 | | 4 | 3 | 5 | |
| 4 | | 2 | 4 | | 5 | | | 4 | | | 2 | |
| 5 | | | 4 | 3 | 4 | 2 | | | | | 2 | 5 |
| 6 | 1 | | 3 | | 3 | | | 2 | | | 4 | |

- estimate rating of movie **1** by user **5**

- The first step is to choose a user-item pair for which to predict rating;
- For instance, we will consider movie 1 and user 5;

# Item-Item Collaborative Filtering

**How to compute similarity:**

**1)** Subtract mean rating $m_i$ from each movie $i$

$m_1 = (1+3+5+5+4)/5 = 3.6$

***row 1:*** *[-13/5, -3/5, 7/5, 7/5, 2/5]*

**2)** Compute cosine similarities between rows

**users**

**movies**

| | 1 | 2 | 3 | 4 | **5** | 6 | 7 | 8 | 9 | 10 | 11 | 12 | **sim(1,m)** |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | | 3 | | ? | 5 | | | 5 | | 4 | | **1.00** |
| 2 | | | 5 | 4 | | | 4 | | | 2 | 1 | 3 | **?** |
| 3 | 2 | 4 | | 1 | 2 | | 3 | | 4 | 3 | 5 | | **?** |
| 4 | | 2 | 4 | | 5 | | | 4 | | | 2 | | **?** |
| 5 | | | 4 | 3 | 4 | 2 | | | | | 2 | 5 | **?** |
| 6 | 1 | | 3 | | 3 | | | 2 | | | 4 | | **?** |

- The next step consists in computing similarity among the selected movie and all the others;

# Item-Item Collaborative Filtering

**How to compute similarity:**

**1)** Subtract mean rating $m_i$ from each movie $i$

$m_1 = (1+3+5+5+4)/5 = \mathbf{3.6}$

***row 1:*** *[-2.6, -0.6,1.4,1.4, 0.4]*

**2)** Compute cosine similarities between rows

**users**

**movies**

| | 1 | 2 | 3 | 4 | **5** | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | | 3 | | ? | 5 | | | 5 | | 4 | |

Subtract mean rating $m_i$ from each movie $i$

$m_1 = (1+3+5+5+4)/5 = 18/5$

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **1** | -13/5 | | -3/5 | | ? | 7/5 | | | 7/5 | | 2/5 | |
| **3** | -1 | 1 | | -2 | -1 | | 0 | | 1 | 0 | 2 | |
| **6** | -8/5 | | 2/5 | | 2/5 | | | -3/5 | | | 7/5 | |

- The next step consists in computing similarity among the selected movie and all the others;
- Next, we select the two most similar movies (3 and 6) rated by user 5;

**Neighbor selection:** Identify movies similar to movie **1**, **rated by user 5**

# Item-Item Collaborative Filtering

For rows 1 and 3, they both have values for users 1, 9 and 11

$$\text{sim}(1, 3) = \frac{(-13/5)(-1)+(7/5)(1)+(2/5)(2)}{\sqrt{(-13/5)^2+(7/5)^2+(2/5)^2}\cdot\sqrt{(-1)^2+(1)^2+(2)^2}} \approx 0.658$$

For rows 1 and 6, they both have values for users 1, 3 and 11

$$\text{sim}(1, 6) = \frac{(-13/5)(-8/5)+(-3/5)(2/5)+(2/5)(7/5)}{\sqrt{(-13/5)^2+(-3/5)^2+(2/5)^2}\cdot\sqrt{(-8/5)^2+(2/5)^2+(7/5)^2}} \approx 0.768$$

# Item-Item Collaborative Filtering - Example

| movies | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | sim(1,m) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | | 3 | | ? | 5 | | | 5 | | 4 | | 1.000 |
| 2 | | | 5 | 4 | | | 4 | | | 2 | 1 | 3 | ... |
| 3 | 2 | 4 | | 1 | 2 | | 3 | | 4 | 3 | 5 | | 0.658 |
| 4 | | 2 | 4 | | 5 | | | 4 | | | 2 | | ... |
| 5 | | | 4 | 3 | 4 | 2 | | | | | 2 | 5 | ... |
| 6 | 1 | | 3 | | 3 | | | 2 | | | 4 | | 0.768 |

**Compute similarity weights:**

$s_{1,3}$=0.658, $s_{1,6}$=0.768

- We can hence select the similarity weights corresponding to the two selected items by looking at the similarities previously computed;

# Item-Item Collaborative Filtering - Example

users

movies

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | | 3 | | ? | 5 | | | 5 | | 4 | |
| 2 | | | 5 | 4 | | | 4 | | | 2 | 1 | 3 |
| 3 | 2 | 4 | | 1 | 2 | | 3 | | 4 | 3 | 5 | |
| 4 | | 2 | 4 | | 5 | | | 4 | | | 2 | |
| 5 | | | 4 | 3 | 4 | 2 | | | | | 2 | 5 |
| 6 | 1 | | 3 | | 3 | | | 2 | | | 4 | |

- The last step consists in predicting the rating that user 5 would give to movie 1;

This can be done by computing a weighted average:

$$r_{ix} = \frac{\sum_{j \in N(i;x)} s_{ij}\, r_{jx}}{\sum s_{ij}}$$

# Item-Item Collaborative Filtering - Example

users

movies

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | sim(1,m) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | | 3 | | 2.54 | 5 | | | 5 | | 4 | | 1.000 |
| 2 | | | 5 | 4 | | | 4 | | | 2 | 1 | 3 | .. |
| 3 | 2 | 4 | | 1 | 2 | | 3 | | 4 | 3 | 5 | | .658 |
| 4 | | 2 | 4 | | 5 | | | 4 | | | 2 | | .. |
| 5 | | | 4 | 3 | 4 | 2 | | | | | 2 | 5 | .. |
| 6 | 1 | | 3 | | 3 | | | 2 | | | 4 | | .768 |

**Predict by taking weighted average:**

$r_{1.5}$ = **(0.658*2 + 0.768*3) / (0.658+0.768) = 2.54**

# Pros/Cons of Collaborative Filtering

**+ Works for any kind of item**

- No feature selection needed

**- Cold Start:**

- Need enough users in the system to find a match

**- Sparsity:**

- The user/ratings matrix is sparse
- Hard to find users that have rated the same items

**- First rater:**

- Cannot recommend an item that has not been previously rated

# Pros/Cons of Collaborative Filtering

- **Popularity bias:**
  - Cannot recommend items to someone with unique taste
  - Tends to recommend popular items
- **Ethical and social issues:**
  - Can lead to filter bubbles and radicalization spirals

# Evaluation and Implications

## Compare predictions with known ratings

Root-mean-square error (RMSE)

$$\sqrt{\frac{\sum_{xi}\left(r_{xi}-r^{*}_{xi}\right)^{2}}{N}}$$

where $r_{xi}$ is predicted, $r^{*}_{xi}$ is the true rating of x on i

Rank Correlation:

- Spearman's correlation between system's and user's complete rankings

# Twitter's Recommendation Algorithm

Twitter's Recommendation Algorithm is a set of services and jobs that are responsible for serving feeds of Tweets and other content across all Twitter product surfaces (e.g. For You Timeline, Search, Explore, Notifications)

https://github.com/twitter/the-algorithm

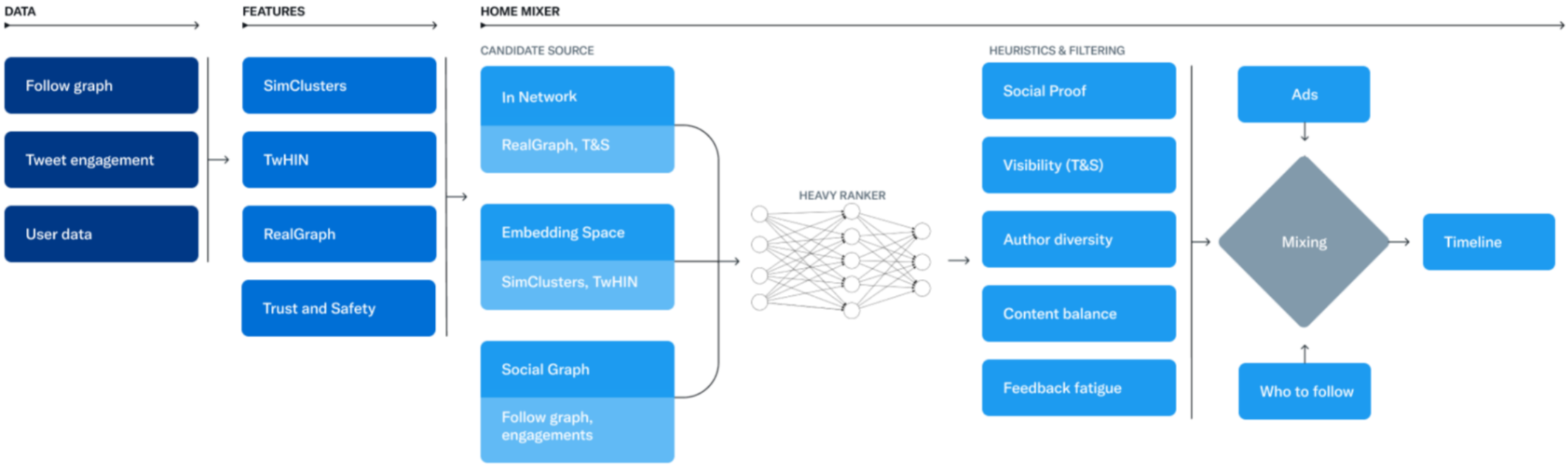
# Twitter's Recommendation Algorithm

## Home Mixer

# Twitter's Recommendation Algorithm

## Home Mixer

# Twitter's Recommendation Algorithm



# Candidate Sources

Find candidates from people you follow (**In-Network**) and from people you don’t follow (**Out-of-Network**).

The For You timeline consists of 50% In-Network Tweets and 50% Out-of-Network Tweets on average, though this may vary from user to user.

# Candidate Sources

## In-Network Source

**Real Graph** is a model which predicts the likelihood of engagement between two users. The higher the Real Graph score between you and the author of the Tweet, the more of their tweets we'll include.

# Candidate Sources

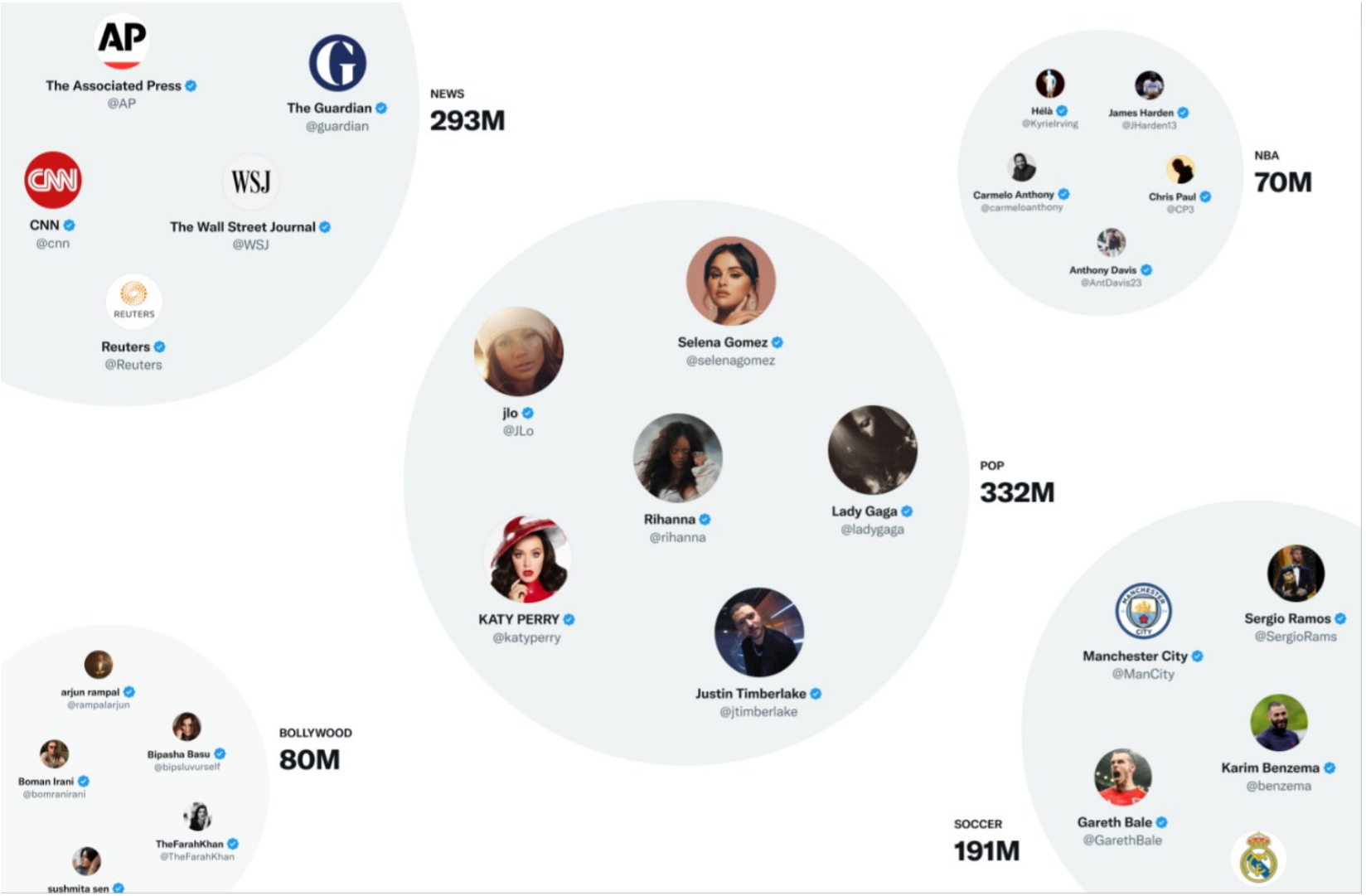
## Out-of-Network Source

**Social Graph** is a model which estimate what you would find relevant by analyzing the engagements of people you follow or those with similar interests.

**Embedding Spaces** to generate numerical representations of users' interests and Tweets' content. Then calculate the similarity between any two users, Tweets or user-Tweet pairs in this embedding space. Provided we generate accurate embeddings, we can use this similarity as a stand-in for relevance.

# Embedding Spaces

**SimClusters**



# Ranking

~1500 candidates that may be relevant.

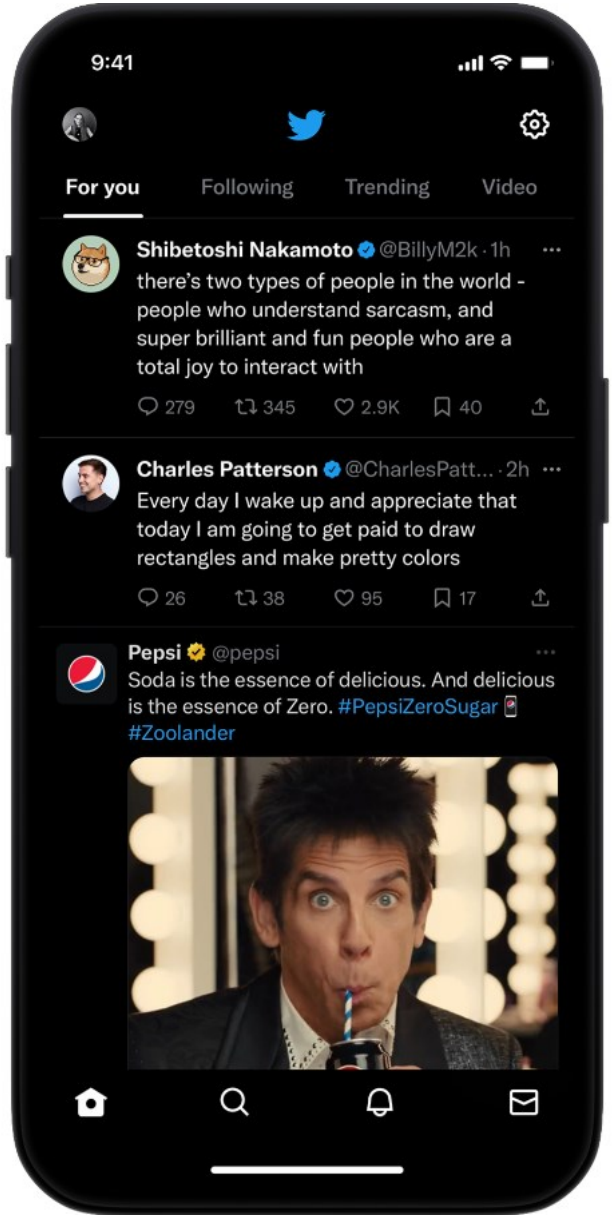
Ranking is achieved with a ~48M parameter neural network that is continuously trained on Tweet interactions to optimize for positive engagement (e.g. Likes, Retweets, and Replies).

# Heuristics, Filters, and Product Features

**Visibility Filtering**: Filter out Tweets based on their content and your preferences. For instance, remove Tweets from accounts you block or mute.
**Author Diversity**: Avoid too many consecutive Tweets from a single author.
**Content Balance**: Ensure we are delivering a fair balance of In-Network and Out-of-Network Tweets.
**Feedback-based Fatigue**: Lower the score of certain Tweets if the viewer has provided negative feedback around it.
**Social Proof**: Exclude Out-of-Network Tweets without a second degree connection to the Tweet as a quality safeguard. In other words, ensure someone you follow engaged with the Tweet or follows the Tweet's author.
**Conversations**: Provide more context to a Reply by threading it together with the original Tweet.
**Edited Tweets**: Determine if the Tweets currently on a device are stale, and send instructions to replace them with the edited versions.

# Mixing and Serving

The system blends together Tweets with other non-Tweet content like Ads, Follow Recommendations, and Onboarding prompts, which are returned to your device to display.



# References

- Chapter 9 from «Mining Massive Datasets» (http://infolab.stanford.edu/~ullman/mmds/ch9.pdf), excluding:
  - Section 9.2.3;
  - Section 9.2.7;
  - Section 9.3.2;
  - Section 9.3.3;
  - Section 9.4;